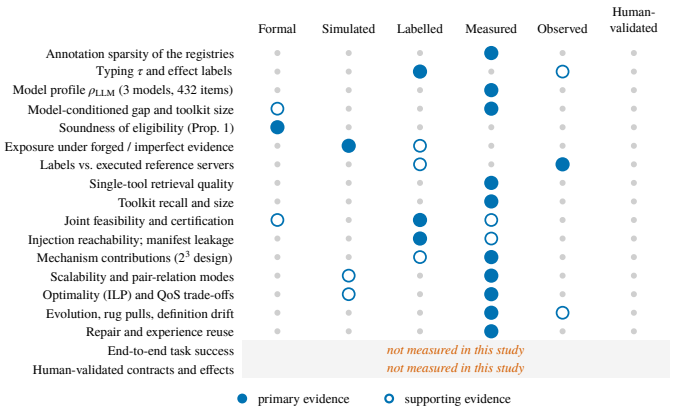


Figure 8. Evidence map: the class of evidence behind each claim evaluated in this section (filled: primary, open: supporting). Shaded rows are not measured.

Table 7. Data, splits and their use.

Data	Size	Use
Merged registry	569 / 76	tools / servers; 44 annotated; all experiments
LiveMCPBench tasks	95	multi-step test tasks (94 with gold tool)
MCPToolBench++ dev	60	tuning of every method (10 per category)
MCPToolBench++ test	300	single-tool retrieval (50 per category, 66 targets)
Security task set	155	95 test tasks + 60 dev prompts, two policies
Calibration suite	432	6 operations × 4 sizes × 2 languages × noise × context; 264 calibration / 168 test items, 3 models
Reference servers	6 / 38	servers / registry tools executed under tracing
Cloned registries	10^3–10^5	near-copies of real contracts (scaling)
Heterogeneous registries	10^3–10^5	generator varying servers, effects, sparsity, vocabulary, duplicates
Cover instances	80	8 families × 10; exact optima by ILP
QoS instances	120	4 roles × 4 providers; 4 weightings × 3 floors

8 Experimental Evaluation

This section evaluates PaMCP along the four research questions of Section 1. Section 8.1 fixes the data, the compared methods, the varied parameters and the statistical protocol. The next five subsections answer RQ1 (representation and model-conditioned gap, Section 8.2), RQ2 (trust under missing, forged and imperfect evidence, Section 8.3), RQ3 (retrieval, toolkit quality and certification, Section 8.4), the security behaviour and the contribution of each mechanism (Section 8.5), and cost, scalability, adaptation and stability (RQ3/RQ4, Section 8.6). Section 8.7 states the limits of the evidence. Every subsection gives the protocol, a results table, figures, the observations and their interpretation, and compares PaMCP with baselines wherever a baseline exists.

8.1 Experimental Setup

Evidence classes. The claims of the paper differ in what can establish them. We label every result with one of six evidence classes: *formal* (a proof in Sections 5–7), *simulated* (generated under stated assumptions, e.g. a verifier with a controlled error rate), *labelled* (scored against machine-generated reference labels), *measured* (obtained from the running implementation against benchmark gold or solver optima), *observed* (from controlled execution traces of real MCP servers) and *human-validated*. Fig. 8 maps each claim to the classes that support it. Two claims have no supporting evidence in this study: end-to-end task success of executed toolkits and human-validated contracts; no conclusion below rests on them.

Registries and tasks. Table 7 lists the data. The registry merges LiveMCPTool [1] and the tool set of MCP-ToolBench++ [2] by `server::name` (76 servers, 569 tools; Fig. 9a). Only 44 tools carry MCP annotations, all of them `readOnlyHint=true`, and one of these is not read-only (Fig. 9c); 119 of the 525 unannotated tools are not read-only. Multi-step tasks are the 95 public LiveMCPBench tasks (94 with a resolvable gold tool). Single-tool retrieval uses MCP-ToolBench++ requests: a 60-request development split (ten per category) for every tuning decision, and a disjoint test sample of 300 requests (50 per category, 66 target tools). Security and ablation experiments use 155 tasks (the 95 LiveMCPBench tasks and the 60 development prompts), each under its least-privilege policy and under a strict read-only policy; confirmatory tests use the test tasks only.

Reference labels. Neither benchmark labels effects or types. Labels J_A (Claude Sonnet, from name, description and schema under a fixed guide) give the typing τ and drive the simulated verifier bound to $h(t)$; labels J_B (Claude Opus, blind, separate guide) are used for scoring (Fig. 9b). They agree on state modification for 558 of 569 tools ($\kappa = 0.944$). Because every tool that is unsafe under J_B (120) is also unsafe under J_A (131), a verifier simulated from J_A can never admit a tool that J_B calls unsafe; we therefore also report the swapped direction (Section 8.3) and compare both labellings with executed servers. All LLM outputs (analyst goal contracts, MCP-Zero requests, COLT training queries, ToolScope merge judgements, calibration answers) were recorded once and are replayed.

Compared methods. Table 8 lists the methods and what each may use. PaMCP combines three mechanisms that the ablations switch independently: the model-conditioned gap (G, Algorithm 1), provenance-sound eligibility (P, Section 5) and set-level reasoning with certification (S: backward closure, minimum-cost cover, certificate; Section 6). PaMCP-L is the relevance-first variant selected on the development split (lexical candidate ranking; structural matching only as a recorded, uncertain constraint). The baselines are adapted to the common registry and share one dense encoder (TF-IDF with truncated SVD); they are adaptations, not the published systems. Every retrieval method was tuned on the development split with a comparable grid (Table 9).

Varied parameters. Table 9 lists every parameter that an experiment varies, with its range and default; parameters not listed keep the values of Section 6.

Statistical protocol. Request-level metrics are resampled with target tools as clusters and task-level metrics with tasks as clusters; paired comparisons use two-sided sign-flip permutation tests on cluster sums, proportions use Wilson intervals, and all intervals are 95%. Two confirmatory families were fixed before the data they test were seen: seven comparisons F1–F7 (before the set-level experiments) and an amendment H1–H5 (committed, dated, before any test-set run of the revised analy-

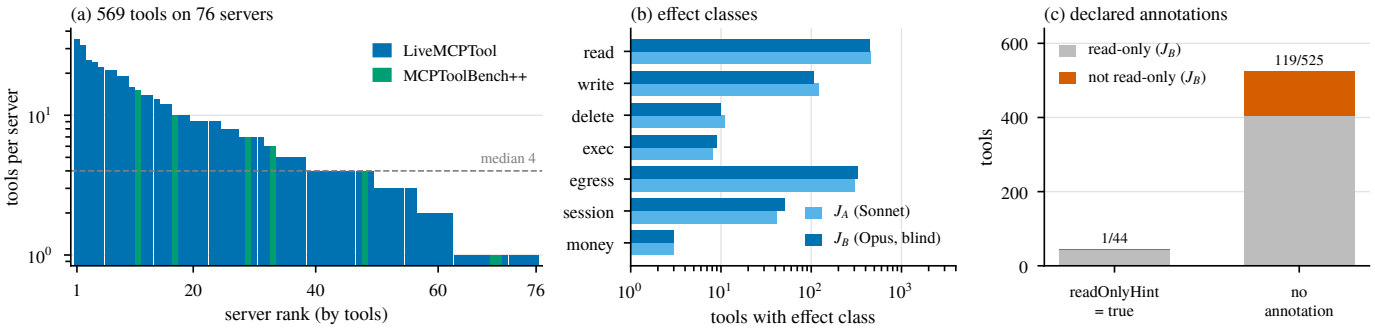


Figure 9. The evaluation registry. (a) Tools per server (log scale). (b) Effect classes under the two LLM labellings. (c) Declared annotations against the read-only status under J_B (numbers: not read-only / total).

Table 8. Compared methods: information used and tuning on the development split.

Method	Information used	Grid
PaMCP	analyst goal contracts, typing τ, evidence, profile, Eq. (6)	14
PaMCP-L	as PaMCP; lexical ranking first, types as constraint	(sel.)
RAG-MCP [3]	raw request, dense top-k	1
BM25 / RRF fusion	raw request, lexical / lexical+dense	12 / 4
MCP-Zero [4]	model-written requests, server routing	4
COLT [5]	query–scene–tool graph of 450 training queries	16
ToolScope [6]	LLM-confirmed tool merging, analyst subqueries	3
CMTF [7]	typed-state search, one tool per step	4
Per-subtask	analyst subgoals, one tool each	–
Full catalogue	all 569 tools	–

Table 9. Parameters varied in the experiments (default in bold).

Parameter	Values	Section
Candidate budget K	1, 5, **10**, 20, 40	8.4, 8.6
Relevance weight w_{rel}	0, 1, 3, **10**	8.4
Profile threshold τ	0.6, 0.75, **0.9**; risk targets 0.05, 0.10	8.2
Input size n	10, 30, 40, 160, 300, 640, 3k, 30k	8.2
Forging servers δ	0, 0.05, 0.1, 0.2, 0.3 (5 seeds each)	8.3
Verifier false accept φ	0, 0.01, 0.02, **0.05**, 0.1, 0.2	8.3
Probe coverage κ	0.25, 0.5, 0.75, **1**	8.3
Trust threshold $\ell_{\min}$	0.2–0.95 (step 0.05), **0.65**	8.3
Poisoned descriptions	0, 10%, 30% of tools	8.5
Mechanisms G, P, S	on/off (2^3) + 5 components	8.5
Registry size	569, 10^3, 10^4, 10^5 tools	8.6
Pair relation	**eager**, partitioned, lazy	8.6
Cost weights $\beta,\gamma,\delta,\eta,\zeta$	0, 0.25, **0.5**, 1	8.6
Utility floor $u_{\min}$	0, 0.4, **0.5**, 0.6, 0.8, 0.9	8.6
Node budget (exact search)	**20,000**	8.6

ses), each with Holm correction and, for recall, a non-inferiority margin of 0.05. All other comparisons are exploratory and reported as such. Code, recorded LLM outputs, per-item results, run manifests (commit, input and label hashes, package versions) and a notebook that re-runs every experiment on a CPU runtime are available.[1]

8.2 RQ1: Representation and Model-Conditioned Gap

RQ1 asks whether the capability model can represent what tools and models do well enough to decide what a request needs. We

[1] `https://github.com/msellamiTN/taas-provenance-discovery`, branch `research/pamcp-final-evaluation`.

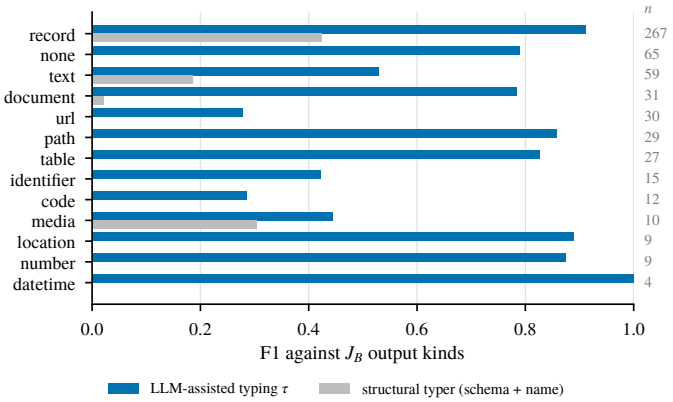


Figure 10. Typing agreement with J_B per output kind (kinds with at least four tools; right: support). The LLM-assisted typing τ is compared with a structural typer that uses schemas and names only.

test three of its ingredients: the typing τ on which structural reasoning relies, the measured reliability profile ρ_{LLM} that decides which operations the model keeps, and the gap of Algorithm 1 that turns the two into a toolkit.

Protocol. Typing is scored against the blind labels J_B (labelled evidence). The profile is measured on a calibration suite of 432 generated data operations (extraction, classification, counting, approximate sums, exact computation and joins; 10 to 640 records; English and French; clean or noisy text; isolated or embedded in a multi-step context). Claude Haiku, Sonnet and Opus answered every item by reading only, without tools, and stated beforehand, per cell, their probability of answering correctly. The threshold τ is chosen on 264 calibration items as the smallest value whose selective risk, the false-native rate among operations kept native, is at most the risk target, and is evaluated on 168 held-out items (measured evidence). One Opus batch of 45 items was interrupted by a usage limit; its items are excluded for Opus, not counted as errors. The gap is exercised on the review scenario of Fig. 1 for growing inputs.

Typing. The LLM-assisted typing agrees with J_B on 78.9% of output kinds ($\kappa = 0.718$, macro-F1 0.704) but on only 35.7% of complete types (kind and domain). A structural typer working from schemas and names alone reaches 22.8% ($\kappa = 0.070$, macro-F1 0.062) and recognises only records, text, documents and media at all (F1 at most 0.42; Fig. 10); τ itself is weakest

Table 10. Model-profile calibration: accuracy (Wilson 95% CI), self-assessment calibration, and the held-out behaviour of the threshold τ chosen for a 5% risk target (measured profile) versus the 64-item profile at $\tau = 0.9$. Cov.: share of operations kept native; FNR: false-native rate (errors among them).

				Self-assess.		Measured, risk 5%			64-item	
Model	n	Accuracy (%)	n=640	ECE	Brier	τ	Cov.	FNR	Cov.	FNR
Haiku	432	86.1 [82.5, 89.1]	67	0.39	0.30	0.21	83	3.6	21	0.0
Sonnet	432	91.4 [88.4, 93.7]	62	0.20	0.11	0.42	90	3.3	26	4.5
Opus	387	100.0 [99.0, 100.0]	100	0.38	0.20	0.90	100	0.0	–	–

on URLs, code and identifiers (F1 0.28–0.42). Types are therefore reliable at the kind level and unreliable at the domain level, which is where structural matching discriminates; this limits structural retrieval (Section 8.4).

Capability boundaries. Table 10 and Fig. 11 show that the boundary between native and tool-requiring operations depends on the model and on the operation, not only on the input size. Haiku answers 86.1% of the items (95% CI 82.5–89.1%) and fails counting already at medium sizes (21% at $n = 40$, 6% at $n = 160$); Sonnet answers 91.4% (88.4–93.7%) and loses joins as they grow (71% at $n = 40$, 44% at $n = 160$); both fail approximate sums over 640 numbers. Opus answers all 387 items it was given, so its boundary lies beyond the suite. Language matters little (Haiku 88.0% in English against 84.3% in French; Sonnet 91.2% against 91.7%), whereas noise lowers Sonnet from 95.4% to 87.5% and exactness lowers Haiku from 92.5% (tolerant) to 78.1% (exact). A single profile per model family would thus misplace the boundary in both directions.

Measured against self-assessed reliability. All three models underestimate themselves (Fig. 12a): their stated probabilities average 0.47, 0.72 and 0.62 against accuracies of 0.86, 0.91 and 1.00 (ECE 0.39, 0.20 and 0.38). With the measured profile and a 5% risk target, the held-out false-native rate stays within the target for every model (3.6%, 3.3% and 0.0%) while 83%, 90% and 100% of the operations remain native (Fig. 12b). The 64-item profile used before this revision is safe but wasteful at $\tau = 0.9$: it keeps only 21% (Haiku) and 26% (Sonnet) of the operations native and sends most of the work the models perform correctly to tools. Self-assessment cannot replace measurement: at the same target it keeps 24% of Haiku's operations native. On the test and development tasks the measured profile overrules 12 of the analyst's 128 claims that an operation is native (9.4%).

Model-conditioned gap. On the review scenario the gap behaves as designed (Fig. 12c): with tolerant aggregation two tools suffice at $n = 30$ and 300, where the model keeps 60% of the goals, and six are needed at $n = 3{,}000$; exact aggregation adds two tools at the small sizes; at $n = 30{,}000$, and at $n = 3{,}000$ with $\tau = 0.6$, the engine abstains instead of returning an unsupported toolkit. Without gap analysis it abstains in every setting, because no registry tool serves the scenario's interpretive goals.

Interpretation. The representation supports model-conditioned discovery: what a request needs depends on the deployed model, and a measured profile with a risk-targeted threshold bounds the false-native rate on held-out operations at the chosen level. Two limits remain. The suite covers six data operations, not the whole transformation vocabulary of Definition 6, and Opus reaches its ceiling; profile stability across model updates was not measured. Typing is the weaker ingredient: its domain-level disagreement propagates to every structural decision evaluated next.

Table 11. Encodings of a read-only policy over 273 requests with read-only gold tools: eligible share of the registry, unsafe tools admitted (of 120, J_B), exposure and utility of the top five for PaMCP's retrieval stage and MCP-Zero, and exposure when 20% of the servers forge and launder their metadata (verifier false-accept rate 5%, mean of five seeds).

			PaMCP retr.		MCP-Zero		δ=0.2
Encoding	Elig.	Unsafe	Exp@5	Util@5	Exp@5	Util@5	Exp@5
no filter	100.0	120	42.5	49.8	26.4	76.6	43.7
MCP default (display only)	100.0	120	42.5	49.8	26.4	76.6	43.7
closed-world annotations	7.7	1	31.5	0.4	0.0	0.4	58.7
lexical cues	67.8	21	9.9	41.0	12.5	65.6	8.8
verified evidence (P)	77.0	0	0.0	46.2	0.0	72.2	5.5
observed + lexical fallback	77.0	0	0.0	46.2	0.0	72.2	5.5
attested only	26.4	0	0.0	16.5	0.0	18.3	5.8

8.3 RQ2: Trust under Missing, Forged and Imperfect Evidence

RQ2 asks whether provenance-sound eligibility (P) keeps unsafe tools away from the model when metadata is missing, forged or verified imperfectly, and what this costs in utility. Proposition 1 guarantees soundness only if safety-relevant evidence is truthful (formal); the experiments below measure how exposure behaves when that assumption is relaxed (simulated), whether the result depends on how the reference labels were built (labelled), and whether the labels agree with what real servers do when executed (observed).

Protocol. A read-only policy is applied to the 394 requests of the single-tool test sample and the development split; Exp@5 is the share of the 273 requests with read-only gold tools whose top five contain a tool that is not read-only under J_B, and Util@5 the share whose top five contain a gold tool. Each encoding of the policy is applied to PaMCP's retrieval stage and to MCP-Zero. Forgery is simulated by letting a share δ of the servers declare `readOnlyHint=true` on every tool and launder their descriptions (five seeds per δ); the verifier is simulated with false-accept rate φ, false-reject rate ψ and probe coverage κ. For the observed comparison, the six MCP reference servers of the registry were started from their published packages (current versions) in a sandbox; each of their 37 registry tools was called once with schema-derived arguments under system-call tracing and file-system snapshots, and a state change was recorded when the call wrote, renamed or deleted user files, the server's store or version-control content.

Declared metadata is not evidence. Table 11 compares the encodings at full verification coverage. Without a filter, and under MCP's default of displaying annotations without enforcing them, 42.5% of the requests receive an unsafe tool among PaMCP's top five (26.4% for MCP-Zero). Treating the annotations as a closed world keeps only the 44 annotated tools; the one unsafe tool among them then reaches 31.5% of the lists while utility collapses to 0.4%. Lexical cues admit 21 of the 120 unsafe tools (Exp@5 9.9%, Util@5 41.0%; they flag 82.5% of

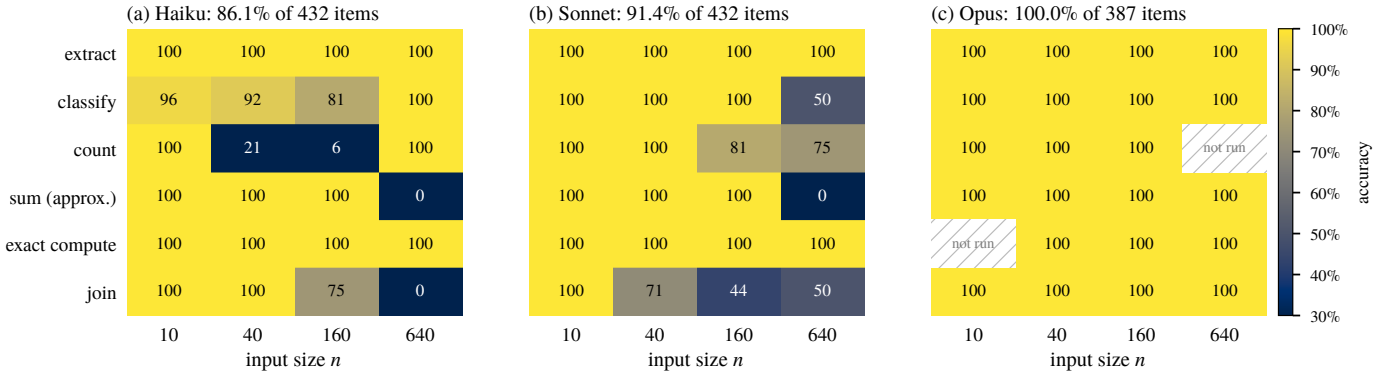


Figure 11. Measured reliability per operation and input size for three models (calibration suite, both splits; cells average languages, noise and context). Hatched cells belong to the interrupted Opus batch.

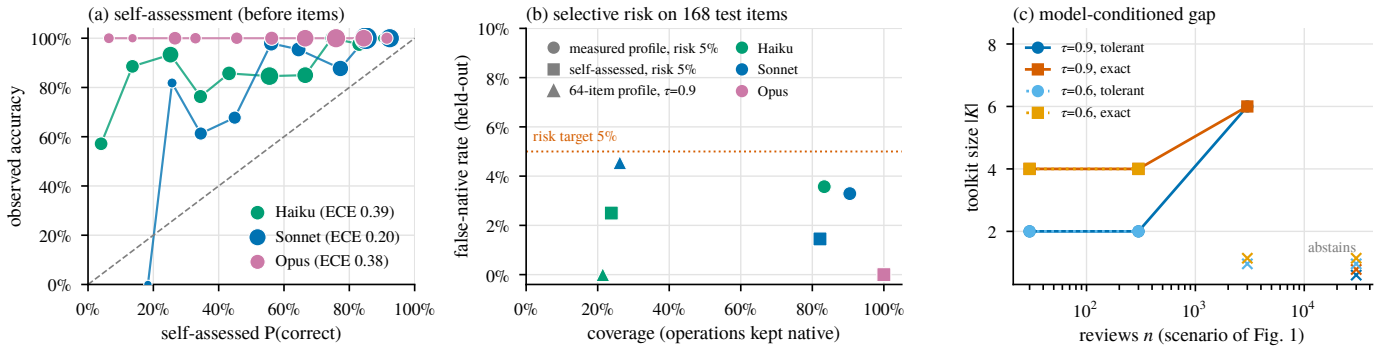


Figure 12. (a) Reliability diagram of the models' self-assessment, given before any item was seen (marker area: items per bin). (b) Held-out coverage and false-native rate of three profiles for each model. (c) Toolkit size of the review scenario against the input size, Sonnet profile; crosses mark abstentions.

the unsafe tools at a precision of 0.54). Verified evidence admits none, keeps 77.0% of the registry eligible and costs 3.6 points of utility for PaMCP's retrieval (46.2% against 49.8%) and 4.4 points for MCP-Zero (72.2% against 76.6%); with lower probe coverage, exposure stays at zero and utility follows the eligible share (Fig. 13a).

Forged and imperfect evidence. Forgery breaks the closed-world reading: its exposure rises from 31.5% to 57.1% when a tenth of the servers forge and to 62.6% at 30% (Fig. 13b). Laundered descriptions barely affect lexical cues (8.7–9.9%), because tool names carry most cues. Verified evidence is independent of δ by construction of the probe: with a 5% false-accept rate its exposure is 5.4–5.6% at every δ, including $\delta = 0$ (five seeds each), at a utility of 46.1–46.7%; which unsafe tools are falsely accepted varies with the seed (0.0–14.7% per seed). Exposure grows roughly linearly with the false-accept rate (1.1%, 5.7%, 9.3% and 15.8% at φ of 1, 5, 10 and 20%; Fig. 13c); errors concentrated on look-alike unsafe tools are not worse. Verified evidence stays below the lexical encoding (9.2%) up to $\varphi = 5\%$ with random errors and up to 10% with look-alike errors: a verifier must be at least this accurate to help. False rejections cost utility instead ($\psi = 0.2$: 46.2% to 39.1%).

Trust threshold. Raising $\ell_{\min}$ from 0.55 to 0.65, the level of observed evidence, lowers exposure from 5.5% to 3.5% and utility from 45.8% to 34.4%; requiring $\kappa \geq 0.8$ as well lowers them to 0.6% and 14.3% (Fig. 14a). Above 0.75 only attested evidence qualifies and exposure rises again to 10.3%, because the few falsely accepted attested tools reach many lists of a pool of 25.4% of the registry. A stricter threshold therefore does not lower exposure by itself; exposure must be read with the eligible pool.

Table 12. State changes observed when executing the 30 exercised tools of six MCP reference servers, against the two labellings and the servers' current annotations. FS: labelled read-only but observed to modify state (false safe; Wilson 95% CI); Unconf.: labelled state-changing, not triggered by the probe.

Reference	TP	FS	Unconf.	TN	False-safe rate (%)	Agree.
J_A (Sonnet)	13	0	0	17	0 [0, 22.8]	100
J_B (Opus)	13	0	0	17	0 [0, 22.8]	100
live annotations	13	0	5	12	0 [0, 22.8]	83

Independence of the reference labels. With the verifier simulated from J_A and scoring on J_B, zero exposure at $\varphi = 0$ is guaranteed by the inclusion of the unsafe sets and carries no empirical content. Swapping the roles removes the inclusion: a verifier simulated from J_B admits the 11 tools that only J_A considers unsafe, and verified evidence then exposes 2.3% of the requests for PaMCP's retrieval, RAG-MCP and ToolScope and 3.9% for MCP-Zero (Fig. 14b), against 9.9% for lexical cues under the original scoring. The protection therefore survives a disagreement between independent labellers, at a non-zero rate that measures their disagreement rather than the verifier.

Observed effects of executed servers. All six reference servers started; 30 of the 37 probed tools completed their call

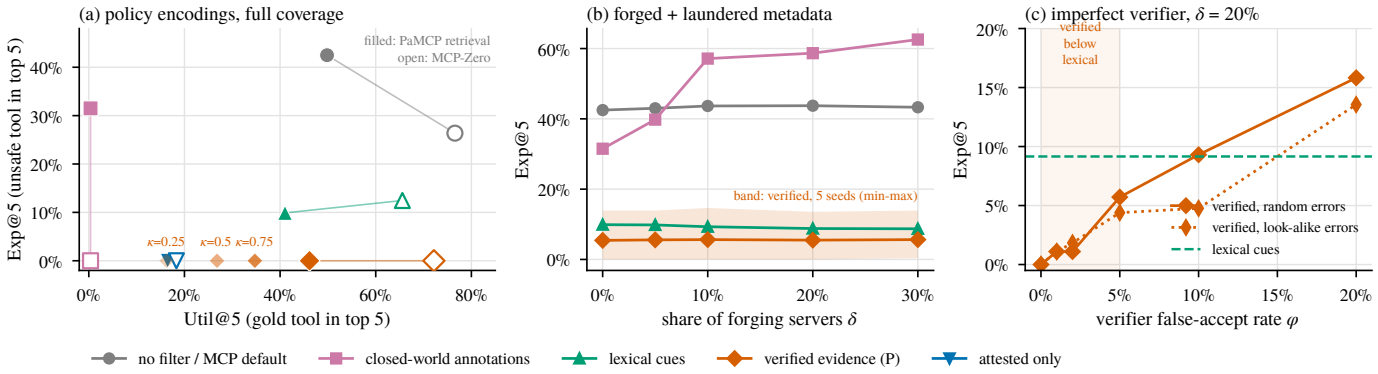


Figure 13. Exposure under a read-only policy (273 requests). (a) Exposure against utility per encoding at full coverage, and verified evidence at lower probe coverage κ. (b) Exposure as a share δ of servers forges annotations and launders descriptions (verifier false-accept rate 5%; mean of five seeds, band: range of the seeds for verified evidence). (c) Exposure of verified evidence against the false-accept rate, with errors at random or on look-alike unsafe tools, against the lexical encoding.

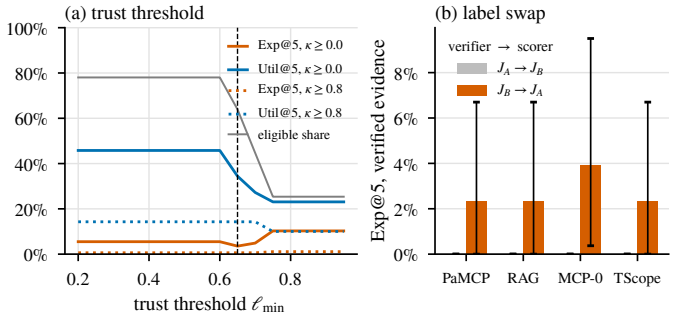


Figure 14. (a) Exposure, utility and eligible share against the trust threshold $\ell_{\min}$ ($\delta = 0.2$, $\varphi = 5\%$), with and without a minimum probe coverage $\kappa \geq 0.8$. (b) Exposure of verified evidence when the verifier is simulated from one labelling and scored on the other (cluster-bootstrap 95% CI).

(six file-system calls received arguments the synthesiser could not make valid, e.g. an edit whose target text is absent, and the fetch call was blocked by the sandbox's network policy), and 13 of them changed state: the memory server's six writers rewrite its store, five Git tools commit, stage, reset, branch or check out, and two file-system tools write or move files (Fig. 15a). Both labellings agree with every exercised call (Table 12): no tool observed to change state is labelled read-only (0 of 13; Wilson upper bound 22.8%), and no label of a state change went unconfirmed. The servers now publish annotations for 25 of these tools, none in the registry snapshot; none claims read-only for an observed state change, and they are conservative on 5 read-only tools. Since the snapshot, every one of the 37 surviving definitions has changed (annotations 25, input schemas 19, descriptions 8; Fig. 15b), one tool was removed and five were added; 13 file-system tools now publish input schemas without properties, which the official Python SDK rejects. Bound to $h(t)$, every piece of evidence about these servers would have been invalidated and re-established before use.

Interpretation. Provenance-sound eligibility removes the exposure that declared or lexical metadata leaves (9.9–42.5%) at a utility cost of 3.6–4.4 points, and degrades gracefully, roughly in proportion to the verifier's false-accept rate, when evidence is imperfect. The zero exposure of an error-free verifier is a property of the simulation; the swapped labels bound the effect of labeller disagreement at 2.3–3.9%, and execution confirms the labels on the state changes of 30 reference tools. These servers are maintained by the protocol's authors and are not adversarial; behaviour against malicious third-party servers is outside the observed evidence (Section 8.7). The drift measurement supports binding evidence to $h(t)$: in the ecosystem studied, definitions change often enough that unbound evidence would be stale.

8.4 *RQ3: Retrieval, Toolkit Quality and Certification*

RQ3 asks whether reasoning over typed capabilities finds better toolkits than retrieval alone. We separate three levels that are often conflated: finding the right single tool (retrieval quality), exposing a small set that covers a multi-step task (gold-tool recall at a given size), and exposing a set whose interfaces, effects and flows are jointly feasible (functional sufficiency, checked by an evaluator independent of the certificate). Executed task success is not measured (Fig. 8).

Protocol. Retrieval is measured on the 300 test requests against benchmark gold (measured), over the 81 MCPToolBench++ tools and over the full registry; every method, including thirteen ranking variants of PaMCP, was configured on the 60 development requests only, and the selection rule was fixed before the test run. The variants isolate the components of Eq. (5) (lexical, dense, structural), relevance-first orderings, confidence-gated or down-weighted structural matching, and oracle types (goal output types or tool output types replaced by J_B), which are diagnostic and not selectable. Toolkits are built for the 94 LiveMCPBench test tasks; group recall counts a task's gold steps (alternative tools of one step count once). Joint feasibility is decided by two independently written evaluators that use J_B and the analyst's goals: a coarse one that compares output kinds and effect classes, and a strict one that also compares domains and collection cardinality, derives required inputs from the schema and treats any egress of private data as a flow violation.

Retrieval: structural matching hurts at scale unless typing is right. Over the 81 MCPToolBench++ tools the relevance-first

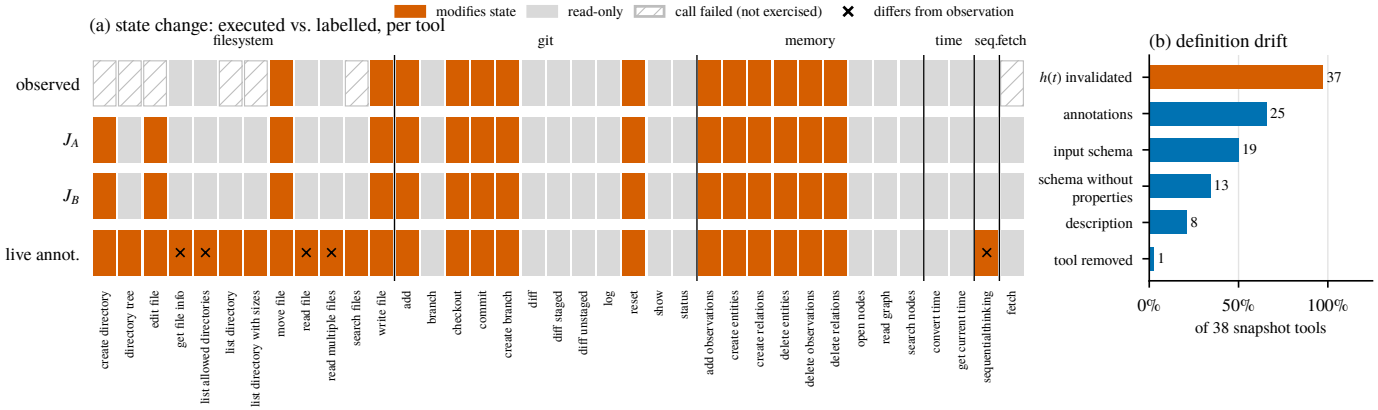


Figure 15. Executed reference servers. (a) Per registry tool: state change observed under tracing, the two labellings and the servers' current annotations (not read-only unless `readOnlyHint=true`). (b) Changes of the live definitions against the registry snapshot; every changed definition changes $h(t)$.

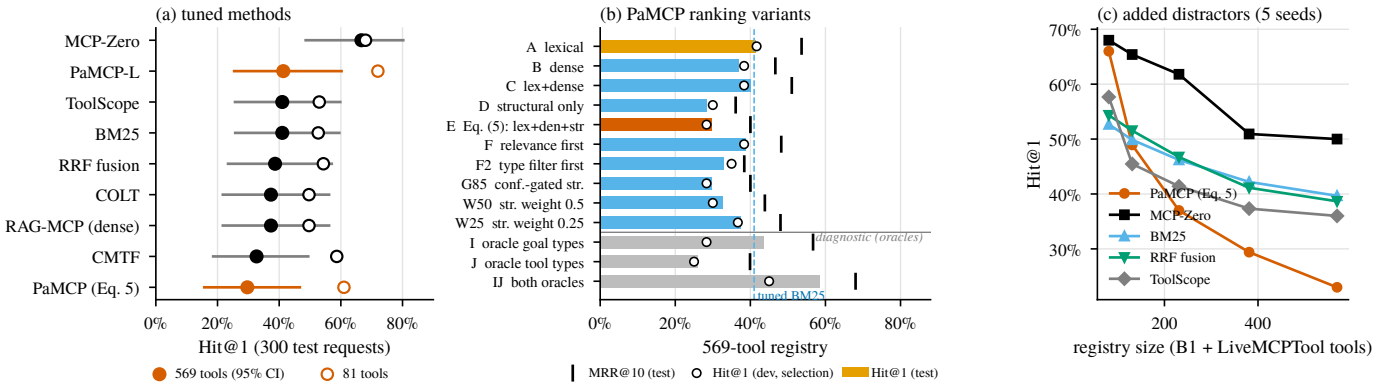


Figure 16. Single-tool retrieval (300 test requests). (a) Hit@1 of the tuned methods over 81 and 569 tools (target-tool clustered 95% CI). (b) PaMCP ranking variants over 569 tools: test Hit@1 (bars), test MRR@10 and the development Hit@1 used for selection; oracle rows are diagnostic. (c) Hit@1 as LiveMCPTool tools are added to the 81 MCPToolBench++ tools (protocol scoring, five seeds).

Table 13. Single-tool retrieval on the 300 test requests with configurations selected on the development split: Hit@1 over the 81 MCPToolBench++ tools and, with target-tool clustered 95% CI, over the 569-tool registry, and the other metrics over 569 tools.

	81 tools	569 tools				
Method	Hit@1	Hit@1 [CI]	Hit@3	Hit@5	MRR	nDCG
BM25	52.7	41.0 [26, 60]	57.3	63.7	0.503	0.543
RAG-MCP (dense)	49.7	37.3 [22, 56]	50.0	54.3	0.451	0.492
RRF fusion	54.3	38.7 [23, 57]	54.7	61.3	0.481	0.525
MCP-Zero	68.0	66.7 [49, 80]	80.0	85.7	0.743	0.775
COLT	49.7	37.3 [22, 56]	50.0	54.3	0.451	0.492
ToolScope	53.0	41.0 [26, 60]	58.3	63.7	0.504	0.543
CMTF	58.7	32.7 [19, 49]	47.0	54.7	0.428	0.488
PaMCP (Eq. 5)	61.0	29.7 [16, 47]	45.7	53.7	0.400	0.461
PaMCP-L	72.0	41.3 [25, 60]	61.3	69.3	0.536	0.603
PaMCP, oracle types (diag.)	71.7	58.3 [38, 74]	75.3	81.3	0.680	0.728

variant PaMCP-L ranks the gold tool first for 72.0% of the requests and the original ranking of Eq. (5) for 61.0%, against 68.0% for MCP-Zero and 49.7–58.7% for the other baselines (Table 13). Over 569 tools the ordering changes: MCP-Zero, whose requests are written by the model for routing, reaches 66.7% (95% CI 48.6–80.2%), PaMCP-L 41.3%, tuned BM25 and ToolScope 41.0%, and Eq. (5) only 29.7%, the lowest of the tuned methods (Fig. 16a). The loss grows with every block of added tools (Fig. 16c). The variants locate the cause (Fig. 16b): structural ranking alone reaches 28.3%, and adding it to lexical and dense ranking lowers Hit@1 from 40.0% to 29.7%; down-weighting it (0.25) or applying it after relevance recovers most of the loss (37.3% and 38.7%), while using it as a filter first does not (33.0%). Confidence gating changes nothing, because every gold tool is typed with confidence of at least 0.85: the typing is confidently wrong at the domain level (Section 8.2). Oracle types show what correct typing would buy: correct goal types alone raise Eq. (5) to 43.7%, correct tool types alone do not help (26.0%), and both together reach 58.3%, above every baseline except MCP-Zero. The dev-selected PaMCP-L is not better than tuned BM25 (confirmatory H1: +0.3 points, $p_{\text{Holm}} = 1.0$) and its gain over Eq. (5), +11.7 points (95% CI 3.0–22.6), is not significant after correction (H2: $p_{\text{Holm}} = 0.070$).

Toolkit size and recall. PaMCP exposes 1.46 tools and 198 tokens per task and reaches a group recall of 0.174 (Table 14); methods that expose more tools reach more gold steps: per-subtask retrieval 0.249 with 2.59 tools, RAG-MCP and COLT 0.293 and 0.295 with five tools and more than 1,000 tokens, ToolScope 0.361 with 5.20 tools. Fig. 17a shows that recall is mostly a function of size: at the same size (1.64 tools) PaMCP reaches 0.189 against 0.130 for RAG-MCP, 0.138 for

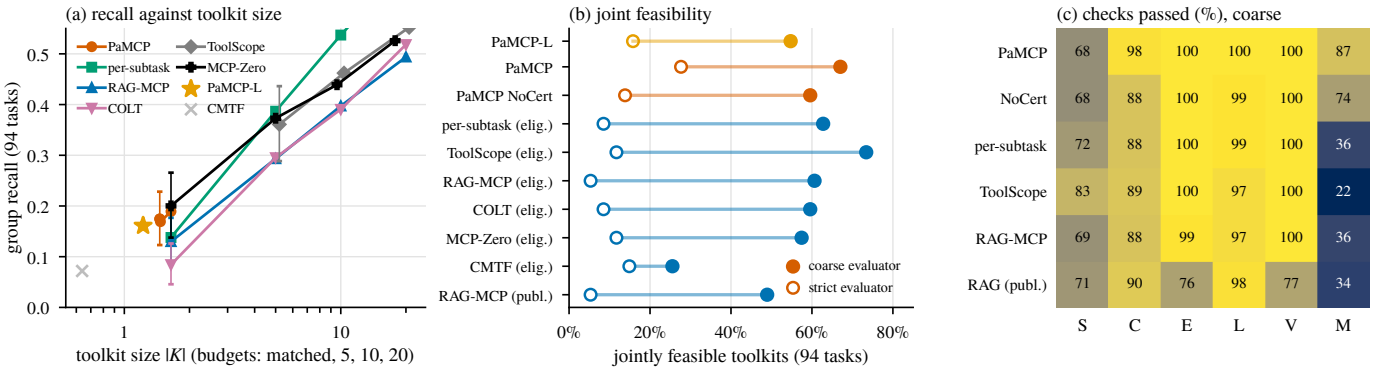


Figure 17. Toolkits on the 94 test tasks. (a) Group recall against toolkit size for candidate budgets (size-matched, 5, 10, 20; bar: 95% CI at the smallest budget). (b) Jointly feasible toolkits for the coarse and the strict evaluator, least-privilege policies. (c) Pass rates of the coarse evaluator's checks: goals served (S), inputs obtainable (C), effects (E), flows (L), evidence level (V) and minimality (M).

Table 14. Toolkits on the 94 LiveMCPBench test tasks: size, manifest tokens, group recall (task bootstrap 95% CI), share of toolkits jointly feasible for the coarse and the strict independent evaluator (least-privilege policies), and abstentions. Baselines restricted to eligible tools unless marked published.

				Joint feas. (%)		
Method	$\|K\|$	Tok.	Group recall [CI]	coarse	strict	Abst.
PaMCP (K=10)	1.46	198	0.174 [0.12, 0.23]	67.0	27.7	7.4
PaMCP-L	1.22	146	0.161	54.7	15.8	9.5
PaMCP NoCert	–	–	–	59.6	13.8	0.0
Per-subtask (elig.)	2.59	346	0.249 [0.19, 0.32]	62.8	8.5	–
RAG-MCP@5 (elig.)	5.00	1244	0.293 [0.22, 0.37]	60.6	5.3	–
COLT@5 (elig.)	5.00	1094	0.295 [0.23, 0.37]	59.6	8.5	–
ToolScope@5 (elig.)	5.20	976	0.361 [0.29, 0.44]	73.4	11.7	–
MCP-Zero (elig.)	1.76	250	0.227 [0.17, 0.29]	57.4	11.7	–
CMTF (elig.)	0.64	105	0.072 [0.04, 0.11]	25.5	14.9	–
RAG-MCP@5 (published)	5.00	1244	0.293 [0.22, 0.37]	48.9	5.3	–

per-subtask retrieval, 0.084 for COLT and 0.200 for MCP-Zero, a difference to RAG-MCP that is not significant (F2: $\Delta = 0.060$, $p_{\text{Holm}} = 0.397$). Non-inferiority of recall to per-subtask retrieval is not established, neither for PaMCP (F1: $\Delta = -0.080$, 95% CI $[-0.166, 0.002]$) nor for PaMCP-L (H3: $\Delta = -0.088$).

Joint feasibility and certification. Under the coarse evaluator, 67.0% of PaMCP's toolkits are jointly feasible; it certifies 92.6% of the tasks and abstains on the rest. Baselines restricted to eligible tools reach 57.4–73.4% (ToolScope highest, CMTF 25.5%), and the difference to per-subtask retrieval is not significant (F3: $\Delta = 0.043$, $p_{\text{Holm}} = 1.0$). The coarse evaluator is, however, too permissive: on ten hand-built cases with ground truth fixed beforehand (same JSON shape with another meaning, domain mismatch, missing required field, array against scalar, scope mismatch, unmet prerequisite, indirect egress of private data, invalid declassification, conditional dependency, a policy violated only by a pair of tools), it decides 2 correctly and the strict evaluator 7; the three remaining cases need scope, credential or declassification information that the labels do not contain. Under the strict evaluator, 27.7% of PaMCP's toolkits are jointly feasible against 5.3–14.9% for the eligible baselines and 13.8% without certification (Fig. 17b; exploratory). Certification acts through obtainable inputs and minimality (coarse C from 88.3% to 97.9%, M from 74.5% to 87.2%, against 36% and 22% for RAG-MCP and ToolScope; Fig. 17c); the binding constraint under both evaluators is goal service (S: 68% coarse, 33% strict), i.e. typing again. The relevance-first PaMCP-L does not improve composition (recall 0.161, strict feasibility 15.8%; H4 against per-subtask retrieval: +7.4 points, $p_{\text{Holm}} = 0.277$).

Interpretation. Set-level reasoning yields small, certified toolkits whose joint feasibility is higher than that of retrieval baselines under a strict independent check, but it does not find more gold tools at equal size, and structural matching lowers single-tool retrieval on the full registry. The oracle results show that the mechanism is not the bottleneck: with correct goal and tool types the structural ranking would be the second best retriever. Typed composition is therefore only as good as the typing it consumes, and the evidence supports PaMCP as a pre-execution feasibility filter, not as a superior retriever. Both evaluators consume LLM labels; neither replaces executing the toolkits.

8.5 *RQ2/RQ3: Security Behaviour and Contribution of the Mechanisms*

This subsection asks two questions. Does the toolkit that PaMCP exposes limit what an injected instruction could make the agent do? And which of the three mechanisms (G, P, S) is responsible for each measured effect?

Protocol. For each of the 155 tasks and each attack class outside the effects the task needs (deletion, tampering, execution, exfiltration, payment, session hijacking), an attack is *reachable* if the exposed toolkit contains a tool that can realise it under J_B (labelled, offline: no injected action is executed). A scope-aware variant distinguishes attacks that are definitely out of the task's scope from those whose scope the labels cannot decide (e.g. a write to an unspecified location when the task needs a scoped write). Poisoning inserts an instruction (a canary) into the description, the tool name or an argument key of 10% or 30% of the tools; the leakage measure is a scan of what each method actually shows the model. The mechanisms are varied in a 2^3 design on the same tasks, registry and scoring, with five components of the full configuration switched off one at a time; effects are estimated with task-level bootstrap intervals and attributed by Shapley values.

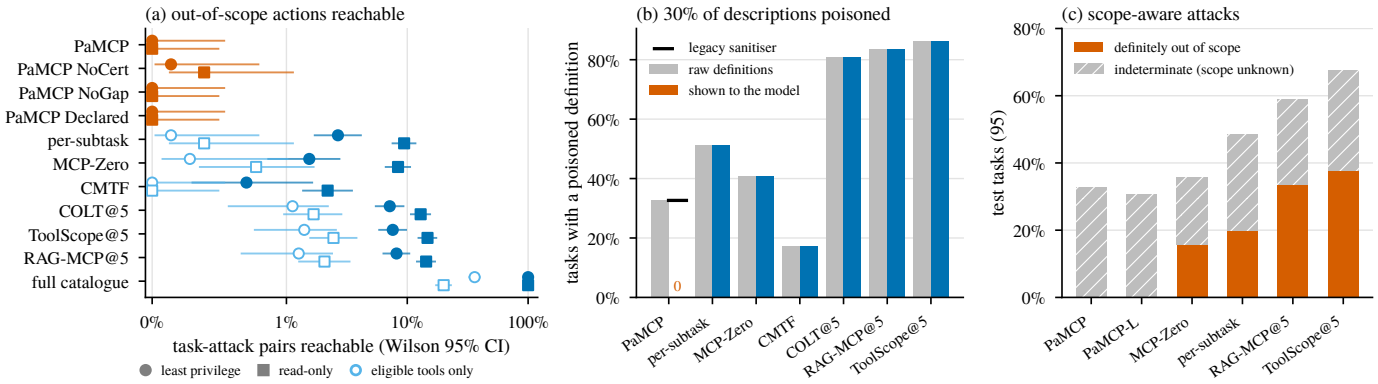


Figure 18. Security behaviour. (a) Task–attack pairs whose out-of-scope action the exposed toolkit can realise, as published (filled) and restricted to eligible tools (open), under least-privilege and strict read-only policies (symmetric log scale). (b) Tasks whose exposed definitions contain an injected instruction when 30% of the descriptions are poisoned: raw definitions of the toolkit against what the method shows the model. (c) Test tasks with an attack that is definitely out of scope or of undecidable scope.

Reachable out-of-scope actions. Under least-privilege policies none of the 712 task–attack pairs is reachable through PaMCP's toolkits, nor through NoGap, Declared or any component variant (Wilson upper bound 0.5%), and one through NoCert, against 8.1% for RAG-MCP, 7.6% for ToolScope, 7.2% for COLT, 2.7% for per-subtask retrieval, 1.5% for MCP-Zero and 0.7% for CMTF as published (Fig. 18a; confirmatory F5 against per-subtask retrieval: $\Delta = -0.027$, $p_{\text{Holm}} < 0.001$). Under a strict read-only policy the baselines reach 8.4–14.7% and PaMCP stays at 0 of 775. Restricting the baselines to eligible tools lowers them to at most 1.4%: most of the protection comes from eligibility, which any retriever can adopt. The scope-aware analysis qualifies this result (Fig. 18c): on the 95 test tasks no PaMCP toolkit admits an attack that is definitely out of scope, against 15.8% for MCP-Zero, 20.0% for per-subtask retrieval, 33.7% for RAG-MCP and 37.9% for ToolScope (H5, PaMCP-L against per-subtask retrieval: $\Delta = -0.200$, 95% CI $[-0.284, -0.126]$, $p_{\text{Holm}} < 0.001$), but 32.6% of its toolkits admit an attack whose scope the labels cannot decide. A write permitted by the policy is therefore not a proof that every write is in scope; deciding it needs scoped effect labels that the benchmarks do not provide.

Poisoned metadata. With 30% of the descriptions poisoned, the raw definitions of PaMCP's toolkits contain an injected instruction in 32.6% of the tasks, and those of the baselines in 17.4% (CMTF) to 86.2% (ToolScope) (Fig. 18b). The aliased manifest, which shows the model only aliases, typed arguments and the certificate, never shows one, in any of the three channels; the earlier sanitiser, which removed descriptions but kept provider names and argument keys, would have shown one in as many tasks as the raw definitions, through the tool name. Of PaMCP's toolkits, 55.5% contain a tool that returns remote content, an indirect-injection channel, but none of them also allows an out-of-scope action, against 3.2–22.6% for the baselines.

Mechanism contributions. Table 15 and Fig. 19 separate the mechanisms. Gap analysis and set-level reasoning each shrink the manifest (main effects -56 and -67 tokens, both $p < 0.001$); they are not additive (G×S $+32$ tokens), and together they take the context from 297 to 176 tokens, with eligibility contributing nothing to size. All three mechanisms lower reachability (main effects -1.0, -1.6 and -1.9 points; Shapley shares 23%, 35% and 42%), from 5.2% of the tasks without any mechanism to none. Recall pays for the reduction: it falls from 0.270 to 0.181, mostly through set-level reasoning (-6.0 points, $p = 0.042$) and its interaction with eligibility (-3.2, $p = 0.018$). No mechanism changes joint feasibility, which stays between 52.9% and 58.1% in every cell. The gap alone does not save significant tokens once the other mechanisms are present (F6, full against NoGap: $\Delta = -16.7$, $p_{\text{Holm}} = 0.269$), but it halves abstentions (10.3% against 20.0%), because goals the model can serve natively need no tool. Among the components, removing backward closure raises abstentions to 12.3%, propagating flow labels only along derivations raises recall to 0.236 at the price of the order-independence of Property 1, and removing the relevance term lowers recall to 0.167; the utility floor and refinement change little.

Table 15. Mechanism ablation on 155 tasks under least-privilege policies: the 2^3 design over the gap (G), provenance-sound eligibility (P) and set-level reasoning (S), and five components of the full configuration. Reach: task–attack pairs whose out-of-scope action the toolkit can realise.

G	P	S	Recall	Tokens	Joint (%)	Reach (%)	Abst. (%)
–	–	–	0.270	297	53.5	1.1	0.0
on	–	–	0.239	208	55.5	0.4	0.0
–	on	–	0.295	316	58.1	0.1	0.0
–	–	on	0.223	221	52.9	0.1	16.1
on	on	–	0.253	229	56.8	0.1	0.0
on	–	on	0.230	189	54.8	0.0	6.5
–	on	on	0.183	193	55.5	0.0	20.0
on	on	on	0.181	176	56.1	0.0	10.3
Full, no backward closure			0.181	173	53.5	0.0	12.3
Full, flow along derivations			0.236	206	60.6	0.0	5.8
Full, no relevance term			0.167	141	54.8	0.0	10.3
Full, no utility floor			0.181	176	56.1	0.0	10.3
Full, no refinement			0.179	175	55.5	0.0	11.0

Interpretation. Under the labelled, offline threat model, PaMCP's toolkits leave no definite out-of-scope action reachable and its manifest never carries an injected instruction, whereas retrieval baselines expose both. The protection is shared between mechanisms: eligibility removes most reachable actions, gap analysis and set-level reasoning shrink the

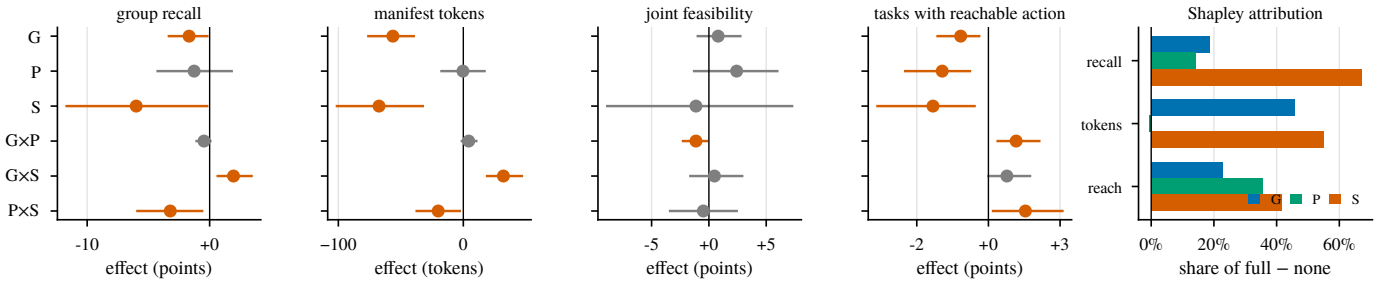


Figure 19. Factorial analysis of the three mechanisms (155 tasks, least privilege): main effects and two-way interactions with task-bootstrap 95% CIs (orange: interval excludes zero), and the Shapley attribution of the difference between the full configuration and the configuration without mechanisms.

Table 16. Engine build time by pair-relation mode (cloned registries; eager at 10^5 from the protocol run) and request latency over 30 tasks (two virtual CPUs); last column: heterogeneous registries, lazy mode.

	Build (s)			Latency (ms)			Het.
Tools	eager	part.	lazy	p50	p95	Abst.	p50
569	2.2	2.6	2.5	49	191	2/30	–
1,000	2.8	3.0	3.0	58	786	3/30	44
10,000	30.3	14.5	14.4	281	5689	9/30	192
100,000	2223.7	134.5	133.4	2669	16098	10/30	3207

exposed context and remove the rest, and the aliased manifest closes the metadata channel. The same mechanisms cost recall, and none of them improves joint feasibility on its own. Whether injected instructions are followed and executed by an agent, and whether runtime enforcement blocks them, requires the execution environment discussed in Section 8.7.

8.6 *RQ3/RQ4: Efficiency, Optimality, Evolution and Stability*

The last question is operational: what does discovery cost as registries grow, how close is the composition to the optimum of Eq. (6), does the QoS objective make the intended trade-offs, and does the engine adapt to registry changes, failed certificates and repeated requests without losing its guarantees?

Protocol. Latency is measured on two virtual CPUs over 30 tasks per registry (measured). Registries of 10^3 to 10^5 tools are built either by cloning the real contracts or by a generator that varies the number of servers, the mix of effects, the share of private outputs (forbidden pairs), annotation sparsity, the type vocabulary and duplication. The pairwise risk relation is built in three modes: eager (all pairs, as in the protocol run), partitioned by effect class, and lazy (decided and cached on demand). Optimality is measured on 80 generated cover instances in eight families against an integer program solved with HiGHS, written independently of the engine. QoS is tested on 120 instances in which four roles each have four interchangeable providers with different price, latency and failure rate, so that all 256 toolkits and their exact Pareto front can be enumerated. Evolution replays the removal and re-insertion of each of the 76 servers; refinement uses the test tasks; experience reuse replays 300 requests against a pattern repository with mandatory revalidation.

Scalability. On the real registry a request takes 49 ms at the median (191 ms at the 95th percentile): eligibility 0.08 ms, per-goal retrieval 45 ms, against 12 ms for one dense ranking of the request (RAG-MCP). The median grows to 58 ms at 10^3 cloned tools, 281 ms at 10^4 and 2.7 s at 10^5 (16.1 s at the 95th percentile), almost all of it retrieval and composition (Fig. 20b); the bitmap test alone takes 0.024 ms over 10^6 contracts, an index measurement that does not extend to the full engine. Building the engine was the bottleneck of the protocol run, which materialises the pairwise risk relation eagerly: 2,224 s at 10^5 tools (Table 16). Partitioning the relation by effect class or deciding it lazily gives identical toolkits on all 94 test tasks and builds in 134 and 133 s at 10^5 tools, 16.5 times faster, with a growth close to linear (Fig. 20a). On heterogeneous registries of 10^4 tools the lazy mode builds in 4.6–5.9 s whatever the factor, while the partitioned mode depends on the density of forbidden pairs (43.9 s when half of the outputs are private) and the eager mode takes 52–72 s (Fig. 20c); median latency stays between 113 and 192 ms. At 10^5 heterogeneous tools the lazy engine builds in 57 s and answers in 3.2 s at the median. Two concurrent workers serve 21.0 requests per second against 11.3 for one.

Table 17. Composition against exact optima (integer program, HiGHS) on 80 generated cover instances, ten per family: infeasible instances, abstentions agreeing with the ILP, exact search equal to the optimum among the instances it solved, node-budget exhaustion, greedy excess cost (mean, max; %) and median times (ms).

Family	Inf.	Abst.	Exact	Budget	Greedy (%)	Exact	Greedy	ILP
base	0	10/10	10/10	0	4.4 (17.2)	31	7	0.9
6 alternatives	0	10/10	10/10	0	5.4 (17.1)	998	10	9.4
depth 3	0	10/10	10/10	0	8.0 (36.6)	186	11	7.3
arity 2	0	10/10	10/10	0	7.2 (20.6)	33	7	6.6
conflicts	0	10/10	10/10	0	12.0 (43.2)	77	8	8.6
shared tools	0	10/10	10/10	0	12.6 (26.8)	370	10	6.8
6 goals	0	10/10	1/1	9	6.9 (21.1)	6703	22	6.1
infeasible	10	10/10	0/0	0	–	3	3	1.1

Optimality. Wherever the exact search finishes, it returns the ILP optimum: in all 61 solved instances, across alternative counts, depths, arities, conflict densities and shared tools (Table 17), and the optimum over the candidate set equals the optimum over all tools, so candidate truncation lost nothing on these instances. All ten infeasible instances are recognised, and abstentions agree with the ILP in all 80. The greedy regime is 4.4–12.6% above the optimum on average and up to 43.2% (conflicts; Fig. 21a). With six goals the exact search exhausts its 20,000-node budget in nine of ten instances after 6.7 s and falls back to the greedy regime, 6.9% above the optimum on average, without any guarantee; the ILP solves every family's

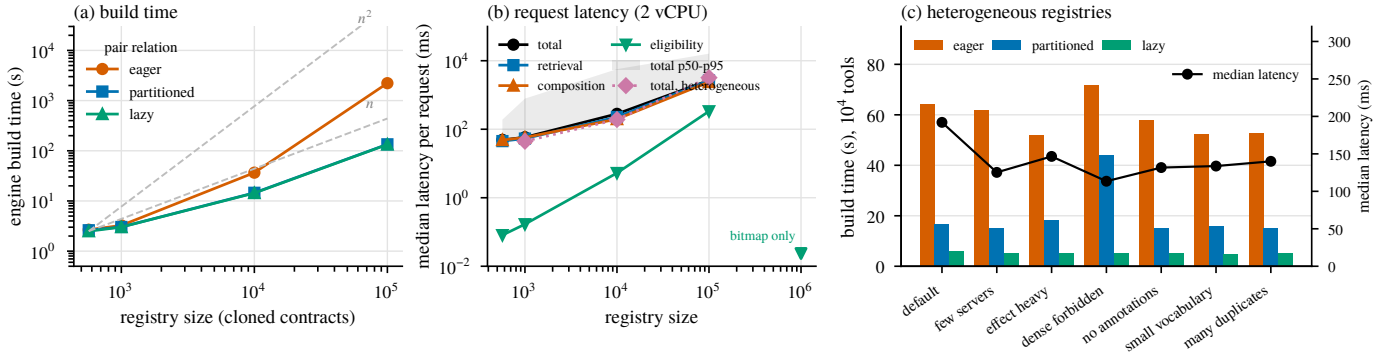


Figure 20. Scalability. (a) Engine build time against registry size for the three pair-relation modes (cloned contracts; dashed: linear and quadratic references). (b) Median request latency by stage (band: total p50–p95), heterogeneous registries, and the bitmap eligibility test alone at 10^6 contracts. (c) Build time and latency at 10^4 tools for each heterogeneous factor.

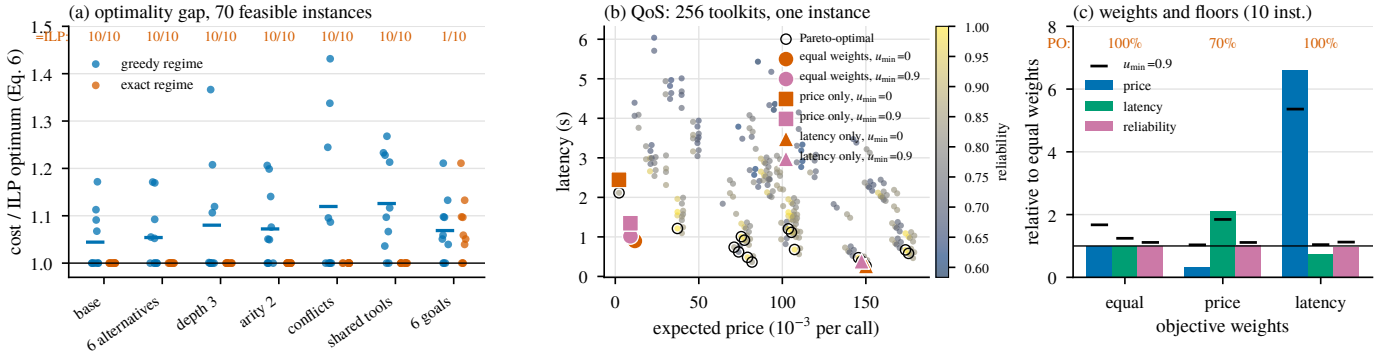


Figure 21. Optimality and QoS. (a) Cost of the greedy and exact regimes relative to the ILP optimum per instance family (top: instances where the exact search equals the optimum; with six goals the exact regime exhausts its budget and falls back to greedy in nine instances). (b) All 256 toolkits of one QoS instance with their Pareto front and the toolkits chosen under three weightings and two utility floors. (c) Mean price, latency and reliability of the chosen toolkits relative to equal weights (bars: $u_{\min} = 0$, ticks: $u_{\min} = 0.9$; PO: share Pareto-optimal).

instances in 1–9 ms at the median, which makes it the better exact back-end for larger covers.

QoS trade-offs. The QoS objective acts as intended once providers differ (Fig. 21b,c). With equal weights every chosen toolkit is Pareto-optimal and meets the utility floor; weighting price alone lowers the mean price to 0.33 times at 2.1 times the latency, weighting latency alone lowers latency to 0.73 times at 6.6 times the price, and a floor of 0.9 raises mean reliability from 0.84 to 0.93 at 1.7 times the price. Two limits appear. With price alone only 70% of the choices are Pareto-optimal, because ties in price are broken without regard to latency; and because Eq. (6) normalises price and latency before weighting them, the equal-weight choice is 32% above the optimum of the same weighted sum in raw units. On the benchmark registries, which publish no QoS data, price and latency weights change no toolkit; the evidence and token weights change 7.2–20.1% and 2.9–7.2% of them, with recall between 0.197 and 0.215.

Evolution. Replaying 152 server removals and insertions, an incremental update takes 13.5 ms at the median (565 ms at the 95th percentile, insertions dominating) against 4.2 s for a rebuild (Fig. 22a). The exact invalidation rule of Proposition 3 invalidates 1.1% of the 35 cached toolkits per event instead of all of them, and after the replay 33 of the 35 recomputed toolkits equal the originals. All 20 simulated rug pulls lose their observed evidence and regain it when the verified definition returns; with the drift measured on the reference servers (Section 8.3) this mechanism would fire on every one of their tools.

Refinement. On the 95 test tasks, 91.6% are certified at once; the deterministic repair operators raise this to 92.6%, repairing one of six χ_1 and neither χ_5 counterexample. With χ_1 induced by a candidate budget of one, they raise certification from 72.6% to 87.4%, repairing 54% of the 24 χ_1 and one of two χ_5 (Fig. 22b); the revenue scenario of Fig. 3 is repaired by the χ_5 split. LLM-proposed repairs were not evaluated.

Experience reuse. Over 300 requests the repository stores 158 patterns and reuses one for 98 requests (32.7%), each saving an analyst call (Fig. 22c). After the mandatory revalidation, 69.4% of the reused toolkits equal the cold ones, certification is unchanged (82.7% against 82.3%), recall on the reused requests is slightly lower (0.184 against 0.214). Revalidation was probed on 89 reused requests by changing the state after storage: removing the pattern's servers, revoking or rewriting its tools' definitions, applying a strict read-only principal, or switching to Haiku's profile; no revalidated toolkit violated the changed state (the stricter principal led to 5 abstentions). Because analyst calls are replayed here, the latency saving of reuse is not visible (18.3 against 17.7 ms); its benefit is the avoided LLM call.

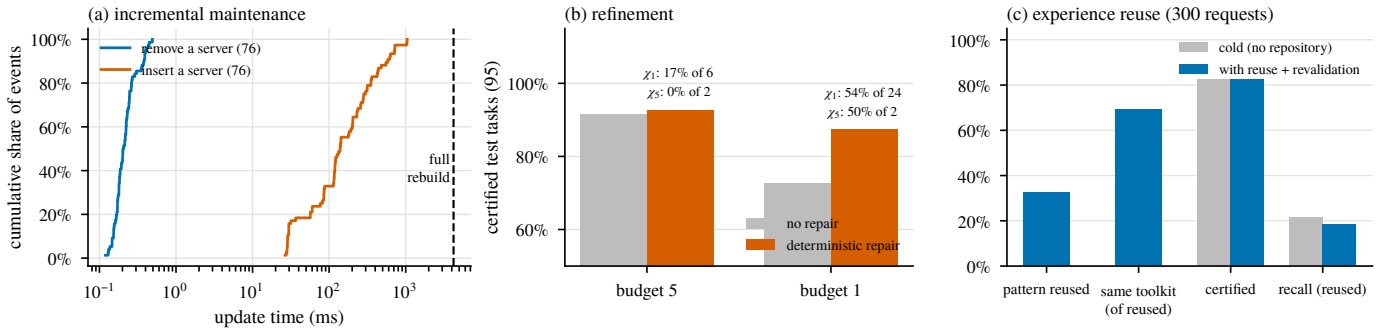


Figure 22. Adaptation. (a) Distribution of incremental update times when each of the 76 servers is removed and re-inserted, against a full rebuild. (b) Certified test tasks before and after the deterministic repair operators, at the default candidate budget and with χ_1 counterexamples induced by a budget of one (labels: share of each counterexample class repaired). (c) Experience reuse with revalidation against cold discovery.

Table 18. Confirmatory comparisons, fixed before the data they test (F1–F7 before the set-level experiments; H1–H5 in a dated amendment before the revised test runs), Holm-corrected within each family. Differences in points (tokens for F6), PaMCP minus comparator, with 95% CI.

	Comparison	Δ	95% CI	p_{Holm}	Verdict
F1	Group recall vs per-subtask (non-inf., 0.05)	-8.0	[-16.6, +0.2]	1.000	not shown
F2	Size-matched recall vs RAG-MCP	+6.0	[-1.6, +13.7]	0.397	n.s.
F3	Joint feasibility vs per-subtask (elig.)	+4.3	[-8.5, +18.1]	1.000	n.s.
F4	Exp@5 verified vs lexical	-9.9	[-13.9, -6.6]	<0.001	supports
F5	Reachable attacks vs per-subtask	-2.7	[-3.9, -1.5]	<0.001	supports
F6	Tokens, full vs NoGap	-16.7	[-34.6, +1.0]	0.269	n.s.
F7	Hit@1 goal contract vs raw RRF (569)	-15.7	[-21.3, -10.0]	<0.001	against
H1	Hit@1 PaMCP-L vs tuned BM25 (569)	+0.3	[-8.9, +8.8]	1.000	n.s.
H2	Hit@1 PaMCP-L vs Eq. (5) (569)	+11.7	[+3.0, +22.6]	0.070	n.s.
H3	Group recall PaMCP-L vs per-subtask (non-inf.)	-8.8	[-17.6, -0.5]	1.000	not shown
H4	Strict joint feas. vs per-subtask (elig.)	+7.4	[+0.0, +14.9]	0.277	n.s.
H5	Definite out-of-scope vs per-subtask	-20.0	[-28.4, -12.6]	<0.001	supports

Statistical stability. With 300 requests on 66 target tools, intervals that resample target tools are 2.7 to 3.3 times wider than request-level ones, so request-level intervals would overstate precision; all intervals above resample clusters. The ranking of the retrieval methods is stable across category-stratified half splits (Kendall's $\tau = 0.68$, the same best method in all 200 splits) and less so across tool-disjoint splits ($\tau = 0.43$, 87.5%); these resamples measure stability of one sample, not replication on new data. Table 18 collects the confirmatory comparisons. Four favour PaMCP after correction: exposure against lexical cues (F4), reachable attacks (F5) and definite out-of-scope attacks (H5), all security outcomes, and none on retrieval or recall. One goes against it (F7, single-tool retrieval with Eq. (5)), non-inferiority of recall is shown for neither variant (F1, H3), and the remaining five are not significant.

Interpretation. The full engine scales to 10^5 tools at second-level latency once the risk relation is not materialised eagerly, which the revision shows to be an implementation choice rather than a property of the model; the exact search is optimal where it finishes but should hand larger covers to an ILP solver; and the QoS objective makes Pareto-optimal trade-offs when the registry provides QoS data, which the benchmark registries do not. Evolution is cheap and exact, deterministic repair recovers induced counterexamples but rarely natural ones, and reuse saves LLM calls without weakening certification.

8.7 Threats to Validity

Construct validity. Effects, types and joint feasibility are scored against labels produced by LLMs, not by human annotators; execution confirms these labels only on the state changes of 30 tools of six reference servers, which are maintained by the protocol's authors and are not adversarial. Reachability of injected actions is decided offline from labels; no injected instruction was executed, and whether an agent follows it, or whether runtime enforcement blocks it, was not measured. Group recall rewards gold tools rather than functional sufficiency, and the strict evaluator, although more accurate than the coarse one on hand-built cases (7 of 10 against 2), lacks scope, credential and declassification information. No toolkit was executed end to end, so task success, argument validity, recovery after execution failures and end-to-end latency are outside the evidence (Fig. 8). QoS effects are shown on generated instances only.

Internal validity. One model family provides the two labellings, the typing, the analyst and the profiles, so their errors may correlate; the label swap bounds the effect of their disagreement but not of shared errors. The verifier is simulated with controlled error rates. Each LLM role was sampled once, and one Opus calibration batch (45 items) is missing. The baselines are adaptations sharing a TF-IDF and SVD encoder (a stronger encoder could not be installed): COLT without encoder fine-tuning, ToolScope without its cross-encoder, MCP-Zero with one round of requests. All retrieval methods were tuned on the same 60 development requests with grids of 1–16 configurations; the variant selected for PaMCP on these requests did not transfer to the test set, a reminder that 60 requests are a small selection sample.

External validity. The registry has 569 tools from two benchmarks, with sparse annotations and no QoS data; larger registries are cloned or generated, and latencies are measured on two virtual CPUs. The provider did not expose exact model version strings; the measured profiles and the reference servers' definitions, which all changed since the benchmark snapshot, may not hold for later versions. *Statistical conclusion validity.* Confirmatory comparisons are few and Holm-corrected within families fixed in advance; every other comparison is exploratory, and resampling one benchmark sample measures stability, not replication.

What remains open. Three experiments would close the main gaps: human annotation of a stratified sample of tools by two annotators, executed multi-step compositions over isolated MCP servers including adversarial ones, and faithful reproductions of the strongest baselines with their original encoders. The released artifact contains the annotation sheet, the probing harness used for the reference servers and the scripts of every experiment above, so that these can be run against the same data and scoring.

REFERENCES

[1] G. Mo, W. Zhong, J. Chen, Q. Yuan, X. Chen, Y. Lu, H. Lin, B. He, X. Han, and L. Sun. LiveMCPBench: Can agents navigate an ocean of MCP tools? *arXiv preprint arXiv:2508.01780*, 2025.

[2] S. Fan, X. Ding, L. Zhang, and L. Mo. MCPToolBench++: A large scale AI agent model context protocol MCP tool use benchmark. *arXiv preprint arXiv:2508.07575*, 2025.

[3] T. Gan and Q. Sun. RAG-MCP: Mitigating prompt bloat in LLM tool selection via retrieval-augmented generation. *arXiv preprint arXiv:2505.03275*, 2025.

[4] X. Fei, X. Zheng, and H. Feng. MCP-Zero: Active tool discovery for autonomous LLM agents. *arXiv preprint arXiv:2506.01056*, 2025.

[5] C. Qu, S. Dai, X. Wei, H. Cai, S. Wang, D. Yin, J. Xu, and J.-R. Wen. Towards completeness-oriented tool retrieval for large language models. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management (CIKM '24)*, pages 1930–1940. ACM, 2024.

[6] M. M. Liu, D. Garcia, F. Parllaku, V. Upadhyay, S. F. A. Shah, and D. Roth. ToolScope: Enhancing LLM agent tool use through tool merging and context-aware filtering. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 34095–34119, 2026.

[7] R. Suresh Babu and L. Ganesh Iyer. ToolChoiceConfusion: Causal minimal tool filtering for reliable LLM agents. arXiv:2606.06284, 2026.